Canvas
2D context

Tvorba internetových aplikácií

RNDr. Richard Ostertág, PhD.
KI FMFI UK Bratislava
ostertag@dcs.fmph.uniba.sk

Sources

- Mozilla Developer Network: Canvas
  - https://developer.mozilla.org/en-US/docs/HTML/Canvas
- Web Hypertext Application Technology Working Group (WHATWG)
  - http://www.whatwg.org/specs/web-apps/current-work/multipage/the-canvas-element.html
- Steve and Jeff Fulton: HTML5 Canvas, 2nd Ed.
  - http://shop.oreilly.com/product/0636920026266.do

The canvas element

- <canvas id="myCanvas" width="300" height="150"></canvas>
  - id attribute is useful for canvas identification in JavaScript
  - width and height are optional
    - default is 300 pixels wide and 150 pixels high
    - can be set using DOM properties from JavaScript
      - this clears the actual content of the canvas
    - can be set using CSS
      - this only scale canvas content to layout size
      - may distort content if not scaled to same aspect ratio
      - introduce blurriness when magnified
- fallback content can be specified inside canvas element:
  - <canvas id="clock" width="150" height="150">

    </canvas>
- the canvas element can be styled similarly as img element
  - margin, border, background, …
  - by default the canvas element is fully transparent
  - styling rules don't affect the actual drawing on the canvas

The rendering context

- the canvas element
  - creates a fixed size drawing surface
  - exposes one or more rendering contexts
    - used to create and manipulate the content on drawing surface
- this lecture will focus on the 2D rendering context
  - other contexts may provide different types of rendering
    - till now there are only 2D ("2d") and 3D ("experimental-webgl") context
    - next lecture will be about the 3D context based on OpenGL ES 2.0
- the drawing surface is initially blank (transparent)
  - to display something a script first needs to access the rendering context
    - by calling getContext() DOM method of the canvas element
    - getContext() takes one string parameter, the type of context.
      - var canvas = document.getElementById("myCanvas");
        var ctx = canvas.getContext("2d"); // **uppercase "2D" does not work**
    - checking for support:
      - var canvas = document.getElementById("myCanvas");
      - if (canvas.getContext) {var ctx = canvas.getContext("2d"); /* draw here */}
        else {/* canvas is unsupported */}
  - and draw on it:
      - ctx.fillStyle = "cyan";
        ctx.fillRect(15, 15, 70, 70);

The canvas coordinate system

- by default 1 unit corresponds to 1 pixel
- coordinate (0,0) (the coordinate system origin) is positioned in the top left corner
- X is classical horizontal axis
  - increases to the right
  - by default comes from 0 to the width
- Y is inverted vertical axis
  - increases in down direction
  - comes from 0 to the height

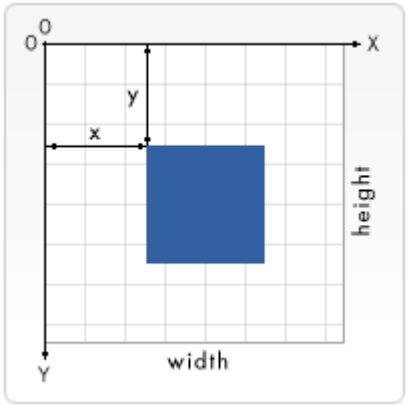


Image from Mozilla Developer Network

Drawing shapes

- canvas supports only one primitive shape:
  Rectangle
- ctx.**fillRect**(x, y, width, height)
  - draws a filled rectangle with no outline
  - (x, y) – the upper-left corner
- ctx.**strokeRect**(x, y, width, height)
  - draws only a outline of the rectangle
- ctx.**clearRect**(x, y, width, height)
  - clears the specified rectangular area
    - makes it fully transparent
- these shapes are not added to the sub-paths list of the current path

Drawing paths – template

- basic template looks like:
  - ctx.beginPath();
    - clears the list of current sub-paths
      - paths are stored as list of sub-paths (lines, arcs, curves, …)
  - ctx.closePath();
    - necessary to close outlines (when using stroke() method)
    - open paths are closed automatically when fill() is called
  - ctx.fill();
    - use current fillStyle
  - ctx.stroke();
    - use current strokeStyle (and lineWidth, lineJoin, …)
- nothing is drawn until call to fill() or stroke()
- when starting new path (e.g. with different color) it is necessary to call beginPath()
  - otherwise new sub-paths will be added to old path
    - old path and new path will be drawn with new fill() or stroke()

Drawing paths – commands

- **moveTo**(x, y)
  - just move the current point to (x, y) in the current path
  - draws nothing
- **lineTo**(x, y)
  - draws line from the current point to (x, y)
  - set the current point to (x, y)
- **arc**(x, y, radius, startAngle, endAngle, anticlockwise)
  - angles are in radians (angle 0 is to the right)
  - to draw circle use: arc(sx, sy, r, 0, 2*Math.PI, true); // or false
  - also draws line from the current point to starting point of the arc
  - set current point to the end point of the arc
- **rect**(x, y, width, height)
  - move the current point to top left corner
    - thus does not connect to previous sub-paths

Drawing paths – Bézier curves

- http://en.wikipedia.org/wiki/Bézier_curve
  - see animations in constructing Bézier curves section
- **quadraticCurveTo**(cp1x, cp1y, x, y)
  - has a start (the current point) and an end (x, y) point
  - just one control point (cp1x, cp1y) (red dot)
- **bezierCurveTo**(cp1x, cp1y, cp2x, cp2y, x, y)
  - cubic Bézier curve uses two control points (cp1x, cp1y) and (cp2x, cp2y)

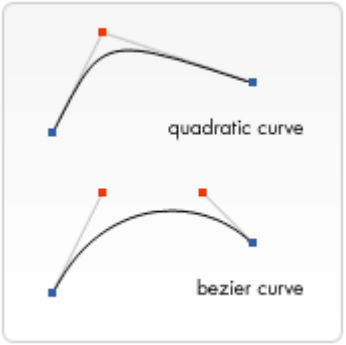


Image from Mozilla Developer Network

Drawing paths – arcTo

- **arcTo**(x1, y1, x2, y2, radius)
- let (x0, y0) be the current point in the path (before arcTo)
  - draw a line from (x0, y0) to the first tangent point on the line from (x0, y0) to (x1, y1)
  - draw an arc from that tangent point to the other tangent point on the line from (x1,y1) to (x2,y2) along the circumference of the circle
  - add the tangent point where the arc ends up, on the line from (x1,y1) to (x2,y2) to the path as the new current point on the path

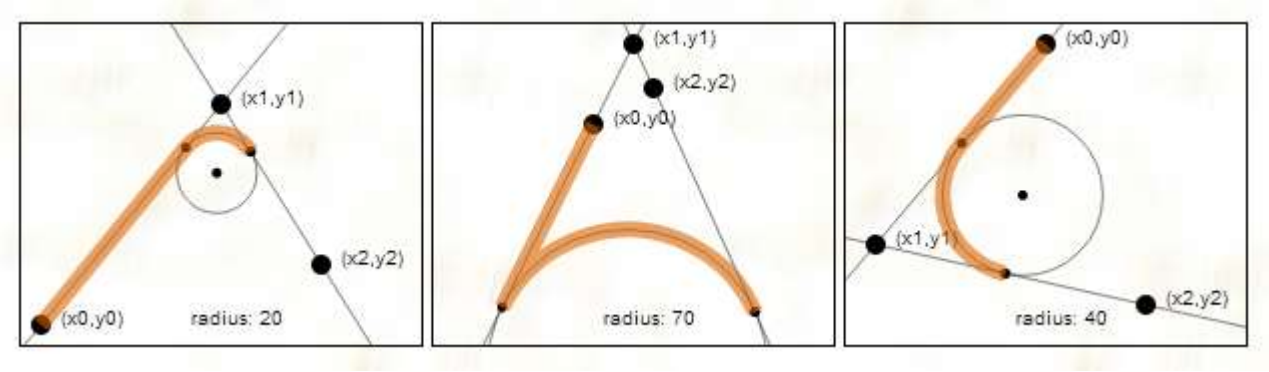
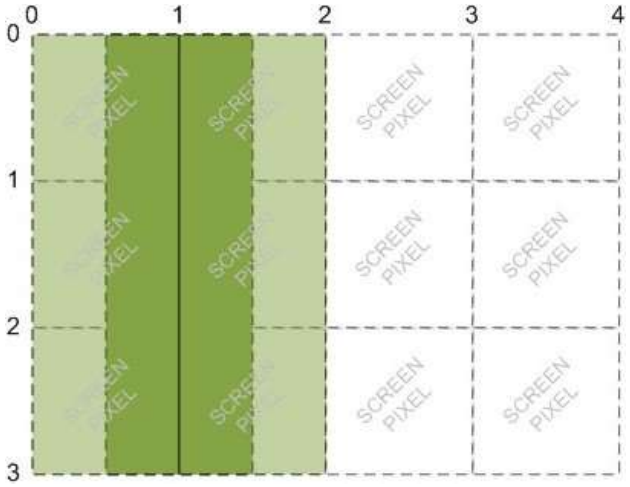


Image and description by Patrick Horgan

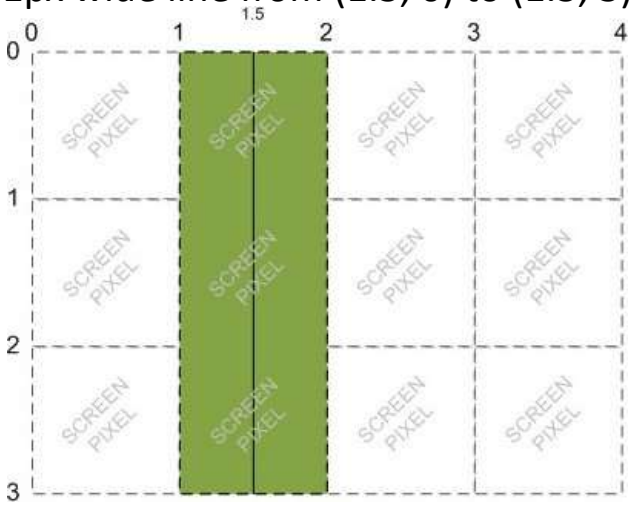
Pixel alignment

- x and y coordinates and lineWidth do not have to be integers
- canvas use antialiasing to draw paths
- this can be lead to blurry lines even if we use only integer values and straight lines, because:
  - whole numbers are corners of pixels
  - coordinates specifies position of axis of the line

1px wide line from (1, 0) to (1, 3)



1px wide line from (1.5, 0) to (1.5, 3)



Images from http://diveintohtml5.info/canvas.html, by Jason Johnson

Fill and stroke (outline) styles

- cxt.**fillStyle** = color|gradient|pattern;
  - color: CSS color value
  - gradient: linear or radial gradient object
  - pattern: pattern object
- cxt.**strokeStyle** = color|gradient|pattern;
  - color: CSS color value
  - gradient: linear or radial gradient object
  - pattern: pattern object

Line styles

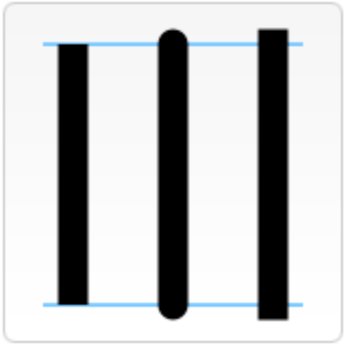


Image from Mozilla Developer Network

- cxt.**lineCap** = "**butt**|round|square";
- cxt.**lineJoin** = "round|bevel|**miter**";
- cxt.**lineWidth** = float; // default 1
- cxt.**miterLimit** = float; // default 10
  - indirectly specifies the max. miter length
  - miterLimit $= 1/\sin\frac{\theta}{2}$
  - miterLimit < 1 is invalid, $= 1$ disables all miters
  - miterLimit $= \sqrt{2} \approx 1.41421$ strips miter for acute angles
  - if the current miter length exceeds the miterLimit, the corner will display as if lineJoin was "bevel“

canvas.toDataURL()

- returns a data URL containing a representation of the image
  - the format can be specified by optional type argument
    - canvas.toDataURL("image/png");
  - defaults to "image/png"
  - type can also be "image/jpeg" or "image/webp"
    - in this case the second argument is treated as indication of image quality (between 0.0 and 1.0)
    - canvas.toDataURL("image/jpeg", 0.75);

Data URL format

- see http://tools.ietf.org/html/rfc2397
- data:[<MIME type>][;charset=<charset>] [;base64],<encoded data>
  - "data:" indicates the this is data URL
  - MIME type indicates type of the content
    - for PNG images if would be "image/png“
    - if not specified, defaults to "text/plain“
  - charset is used only for texts
  - base64 indicates that the content is base64 encoded
    - if omitted, standard URL-encoding is used instead of base64
      - URL-encoding is usually much longer then base64 equivalent

Data URL format example

-
- myImg.src="
AYAAAAf8%2F9hAAAAAXNSR0IArs4c6QAAAAZiS0dEAP8A%2FwD%2FoL2nkwAAAAl
wSFlzAAALEwAACxMBAJqcGAAAAAd0SU1FB9oBDhACHe%2FYZkEAAAIJSURBVDjLn
ZJdSJNhFMf%2Fz%2Fs%2B7z5eP0ZtOmYjh6YtHQwtErOkDxAkukjFAi%2B7MbwJur
Wgi0BI77oRIkyElWCQ3YWjiLKE0IkUQc1pRLUx3VZbc9v7PqeLEGaTVftfHc45v995Lh5
gl9hrj3sNxvIZq7NtqbLK%2FUqS2HUAcB%2B7UrDL%2FmzsrWkdstefut3SPUqSDMY
YKPb1PXs%2B1RsdHH%2Frmp%2BeSb2431co8HaNIBUPeRVjWaClewxaVgNjDCACA
VCMEj261biY3AweyT8obxfh4BxSsbWJ05f89VpWY0LPYG1pApKswFTugBBgeiZWo1p
qp%2BPflqPbnJRvM1c46mQOxhjD%2BvIU3swOwn%2BnA9wggYRAlasdocCkK5%2F
ZIdCyySQAAhGszqPgBjOcTech9N%2FzrWQUnJvS%2BQzfIcilfZHQQqvFfhgWuxf9N35
C6ICuaTConFYXJ0Uul37GGNv9BVvJ8OjLBwMfGMsQkYCW1SF0HbLCsTJ3k2XTif58uE
DQ3nsPPzY%2Bup%2BMdyYMZk5EBCIgEvRjPeC7tvF54eFf%2F8G%2Bg2chcbVtj8Pz
%2BlDnMBSjRI%2FHmlcTkXcH8D%2BpsDbOD4yQ6LjoI4vdc07myr%2FD%2Bz19MK
m2C12Xn5KzqUdHKRm6S2rzyWGqtDXMotRUu058N5VZrxbbkYoN4%2BGVqFG1fSp
ZIHPzl%2Bq6M0UFvwBr1b9fn1OUawAAAABJRU5ErkJggg%3D%3D";

- produces: 

The coordinate system transformations

- all affine transformation can be represented using
  - the homogeneous coordinates $(x, y, 1)$ and
  - the transformation matrix $\begin{pmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{pmatrix}$
  - using equation $\begin{pmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix}$
- allows translation, scaling, rotation and skewing
- composition of transformations ≡ transformations matrix multiplication

Demo program for transformations

JavaScript

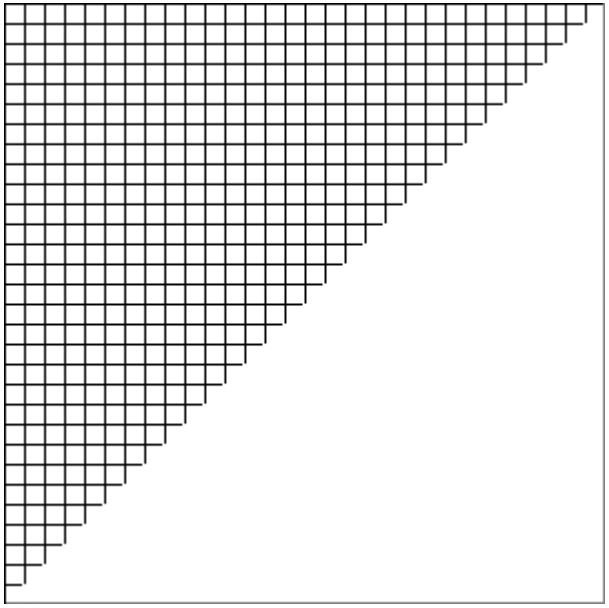
```
var canvas = document.getElementById("myCanvas");
var ctx = canvas.getContext("2d");
ctx.strokeStyle = "black";
ctx.lineWidth = 1;
ctx.strokeRect(0, 0, 300, 300);
ctx.setTransform(1, 0, 0, 1, 0, 0);
ctx.beginPath();
for (var x = 0.5; x < 300; x += 10) {
   ctx.moveTo(x, 0);
   ctx.lineTo(x, 300 - x);
}
for (var y = 0.5; y < 300; y += 10) {
   ctx.moveTo(0, y);
   ctx.lineTo(300 - y, y);
}
ctx.stroke();
myImg.src = canvas.toDataURL();
```

HTML

```
<canvas id="myCanvas" width="300" height="300"></canvas>
<img id="myImg" width="300" height="300" />
```

CSS

```
canvas {
   display: none;
}
```

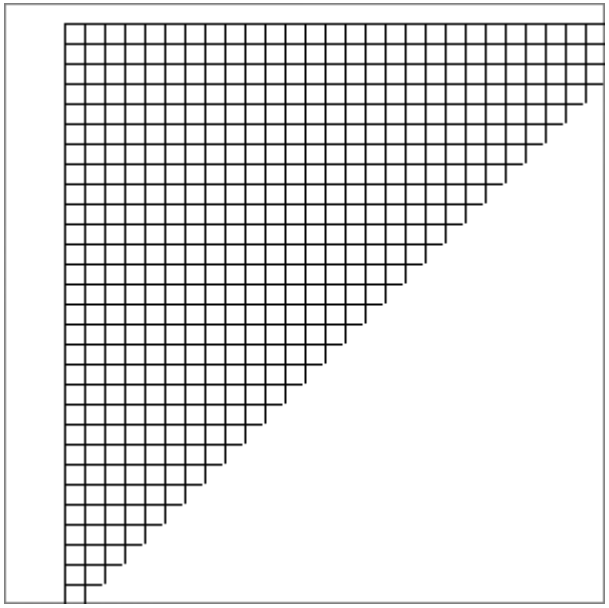


Translation

- translate(t_x, t_y)

$$\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

- ctx.translate(30, 10);
- http://jsfiddle.net/T9v3d/3/

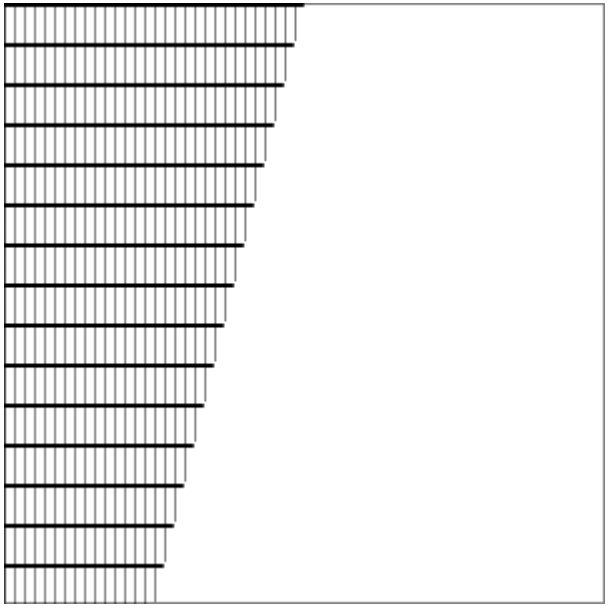


Scaling

- scale(s_x, s_y)

$$\begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} s_x x \\ s_y y \\ 1 \end{pmatrix}$$

- ctx.scale(0.5, 2);
- http://jsfiddle.net/T9v3d/4/

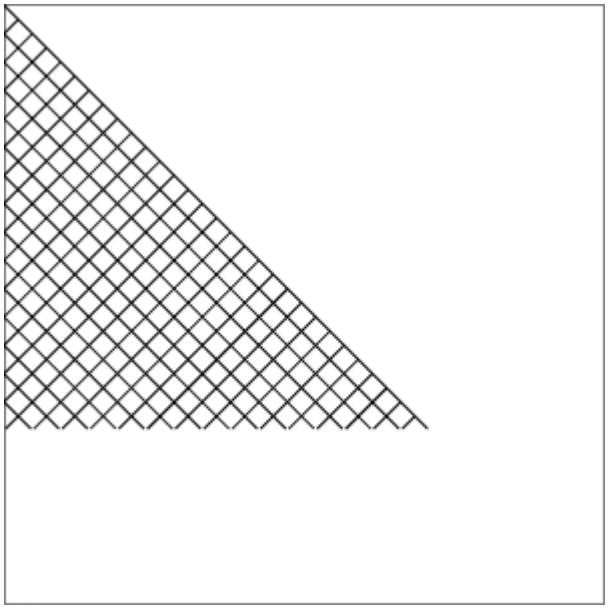


Rotation

- rotate(θ)

$$\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \\ 1 \end{pmatrix}$$

- rotate around the canvas origin (0, 0) by θ radians clockwise
  - to change the rotation center point move the canvas by using the translate method
- ctx.rotate(Math.PI / 4);
- http://jsfiddle.net/T9v3d/5/

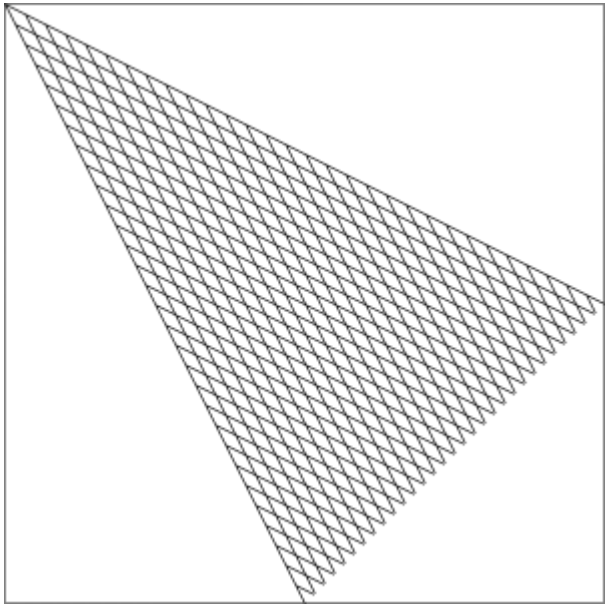


Transformation

- transform(a, b, c, d, e, f)

$$\begin{pmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} ax + cy + e \\ bx + dy + f \\ 1 \end{pmatrix}$$

- setTransform(a, b, c, d, e, f)
  - does not multiply, but set to
- ctx.transform(1, 1 / 2, 1 / 2, 1, 0, 0);
- http://jsfiddle.net/T9v3d/6/

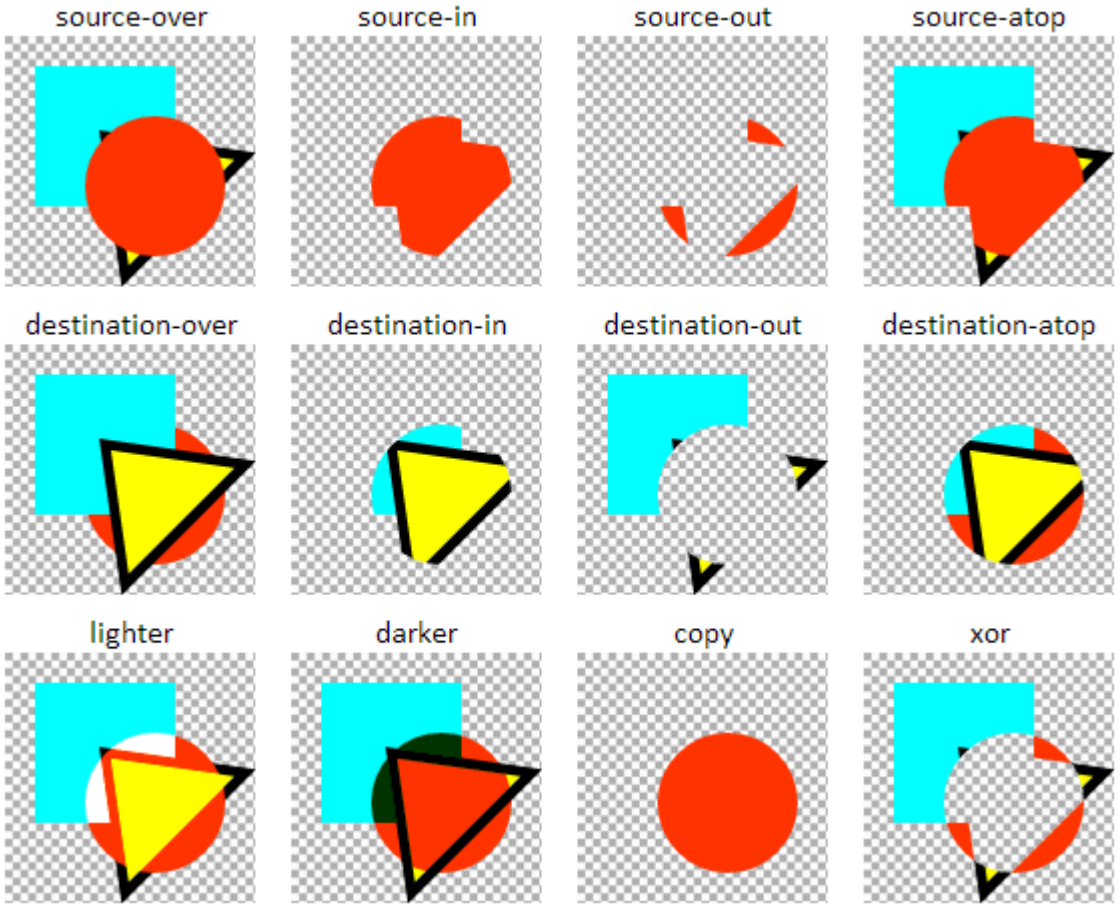


State

- the state includes:
  - the current transformation (such as translate, rotate, …)
  - the current clipping path
  - current values of the following attributes:
    - strokeStyle, fillStyle,
    - lineWidth, lineCap, lineJoin, miterLimit
    - globalAlpha, globalCompositeOperation
    - shadowOffsetX, shadowOffsetY, shadowBlur, shadowColor
    - font, textAlign, textBaseline
- to save or restore the state call
  - ctx.save(); // push current state onto a stack
  - ctx.restore(); // pops state from stack

globalCompositeOperation

default



- draw the rectangle and the triangle
- set **globalCompositeOperation** to specified value
- draw the circle

Shadows

- shadowOffsetX = float
  shadowOffsetY = float
  - indicate offset of the shadow from the object
  - not affected by the current transformation matrix
- shadowBlur = float
  - indicates the size of the blurring effect
  - this value doesn't correspond to a number of pixels
  - not affected by the current transformation matrix
- shadowColor = color
  - value indicating the color of the shadow effect
    - standard CSS color
- applicable to shapes, paths and texts

Gradients

- cxt.**createLinearGradient**(x0, y0, x1, y1);
  - (x0, y0) start point
  - (x1, y1) end point
- cxt.**createRadialGradient**(x0, y0, r0, x1, y1, r1);
  - (x0, y0) center of starting circle with radius r0
  - (x1, y1) center of ending circle with radius r1
- gradient.**addColorStop**(stop, color);
  - stop: a value between 0.0 and 1.0 that represents the position between start and end in a gradient
  - color: CSS color value

Clipping

- ctx.**clip**();
  - use the current path for clipping of future drawing
- once a region is clipped, all future drawing will be limited to the clipped region
  - no access to other regions on the canvas
- you can however save the current canvas region using the save() method before using the clip() method, and restore it with the restore() method any time in the future

Text – properties

- ctx.**font** = "style variant weight size family";
  - same syntax as the CSS font property
    - style: normal, italic or oblique
    - variant: normal or small-caps
    - weight: normal, bold, bolder or lighter
    - size: e.g. **10px**
    - family: e.g. **sans-serif**
- ctx.**textAlign** = "center|left|**start**|right|end";
- ctx.**textBaseline** = "**alphabetic**|top|hanging|middle|ideographic|bottom";

Image from w3schools

Text – methods

- cxt.**fillText**(text, x, y, maxWidth);
  - maxWidth is optional
    - the maximum allowed width of the text in pixels
- cxt.**strokeText**(text, x, y, maxWidth);
  - maxWidth is optional
    - the maximum allowed width of the text in pixels
- ctx.**measureText**(text)
  - returns an object containing the width of the specified text in pixels
  - e.g. ctx.measureText("hello").width

Image drawing

- cxt.**drawImage**(img, x, y);
- cxt.**drawImage**(img, x, y, width, height);
- cxt.**drawImage**(img, sx, sy, swidth, sheight, x, y, width, height);

Working with audio

- HTML5 Canvas can’t manipulate audio
  - but many canvas applications can use that extra dimension of sound
- Audio is represented by
  - HTMLAudioElement object
    - for use with JavaScript
  - <audio> tag
    - for use in HTML5

The <audio> tag

- <audio src="https://.../eg.ogg" controls>
  Your browser does not support the audio element.
  </audio>

- <audio controls autoplay>
  <source src="https://.../eg.ogg" type="audio/ogg">
  <source src="https://.../eg.mp3" type="audio/mp3">
  <source src="https://.../eg.wav" type="audio/wav">
  Your browser does not support the audio element.
  </audio>

- https://jsfiddle.net/RiOS/pss5uafh/

The <audio> tag

- autoplay – the audio will automatically begin playback as soon as it can do so, without waiting for the entire audio file to finish downloading
  - a boolean attribute is considered true if present (even if the value is "false")
- controls – offer controls to allow the user to control audio playback, including volume, seeking, and pause/resume playback.
- loop – automatically seek back to the start upon reaching the end of the audio
- preload – hint to the browser about for the best user experience
  - none: the audio should not be preloaded
  - metadata: only audio metadata (e.g. length) is fetched
  - auto: the whole audio file could be downloaded
  - the empty string: synonym of the auto value
- volume – the playback volume, in the range 0.0 (silent) to 1.0 (loudest)

Audio functions

- load()
  - Starts loading the sound file specified by the src property.
- play()
  - Starts playing the sound file specified by the src property.
    - If the file is not ready, it will be loaded.
- pause()
  - Pauses the playing audio file.

Important audio properties

- duration (in seconds)
  - the total length of the sound
- currentTime (in seconds)
  - the current playing position
- ended (true or false)
  - true if the audio object is at its end
- muted (true or false)
  - silences audio regardless of volume settings
- paused (true or false)
  - whether the audio object is paused
  - set with a call to the pause() function
- autoplay (true or false)
- controls (true or false)
- loop (true or false)
- volume (0.0 to 1.0)

Important audio events

- progress
  - sent periodically to inform about download progress
    - the buffered attribute contains current amount of the audio that has been downloaded
- canplaythrough
  - sent when the entire audio can be played without interruption
    - assuming the download rate remains at least at the current level
- playing
  - sent when the media begins to play
    - for the first time, after having been paused or after ending and then restarting
- volumechange
  - raised when either the volume or the muted property changes
- ended
  - raised when playback reaches the duration of the audio file
    - and the file stops being played

- https://jsfiddle.net/RiOS/ze3n9sfr/

Playing sounds using a single object

- HTML:
  ```
  <audio id="aud" preload>
    <source src="…" type="audio/wav">
  </audio>
  <button id="play">Play</button>
  ```
- JavaScript:
  ```
  play.onclick = function() { aud.play(); }
  ```
- https://jsfiddle.net/RiOS/kdwcew8u/
- Repeatedly press “Play” button
  - the sound is not played every time
  - single element can not play multiple instances

Dynamically creating new sound obj.

- HTML: <button id="play">Play</button>
- JavaScript:

```
function playSound() {
  var sound = document.createElement("audio");
  sound.setAttribute("src", "…");
  sound.play();
}
play.onclick = playSound;
```

- https://jsfiddle.net/RiOS/d1yo3xnz/
- Repeatedly press “Play” button
  - multiple sounds plays simultaneously, but
  - the sound is played with noticeable delay (every time)
  - huge number of sound objects will be created

Solution: reusing preloaded sounds

- HTML:
  ```
  <audio id="aud1" src="…" preload></audio> …
  <audio id="aud5" src="…" preload></audio>
  <button id="play">Play</button>
  <div id="div1">1</div> … <div id="div5">5</div>
  ```
- JavaScript:
  ```
  function setup(n) {
    var sound = document.getElementById("aud" + n);
    var div = document.getElementById("div" + n);
    sound.oncanplaythrough = function() { div.innerHTML = n + ". can play through."; };
    sound.onplaying = function() { div.innerHTML = n + ". sound is playing."; };
    sound.onended = function() { div.innerHTML = n + ". sound ended."; };
  }
  var next;
  for (next = 1; next <= 5; next++) setup(next);
  next = 1;
  play.onclick = function () {
    var sound = document.getElementById("aud" + next);
    next += 1;
    if (next > 5) next = 1;
    sound.play();
  }
  ```
- https://jsfiddle.net/RiOS/5ydbg63L/